UNIVERSIDADE FEDERAL DE MINAS GERAIS
Instituto de Ciências Exatas
Departamento de Ciência da Computação
DCC207– Algoritmos 2
Prof. Renato Vimieiro

Trabalho Prático 1– Indexação Geoespacial e Geometria Computacional Híbrida

Henrique Gomes dos Santos Medeiros – 2021084986
Mateus Souza Barreto - 2023105387

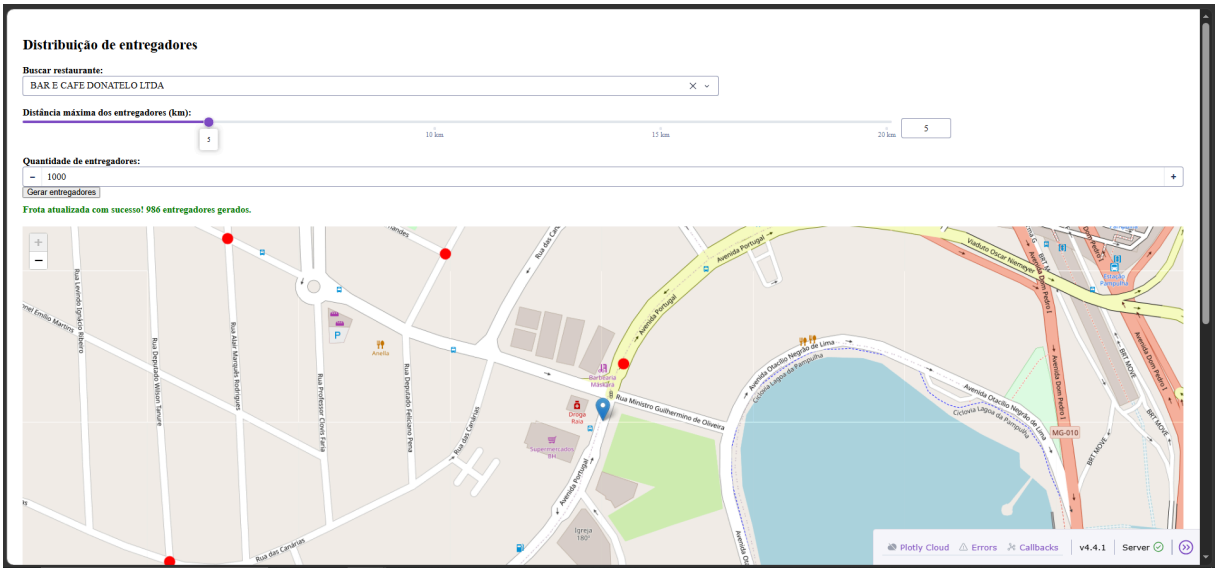
Introdução

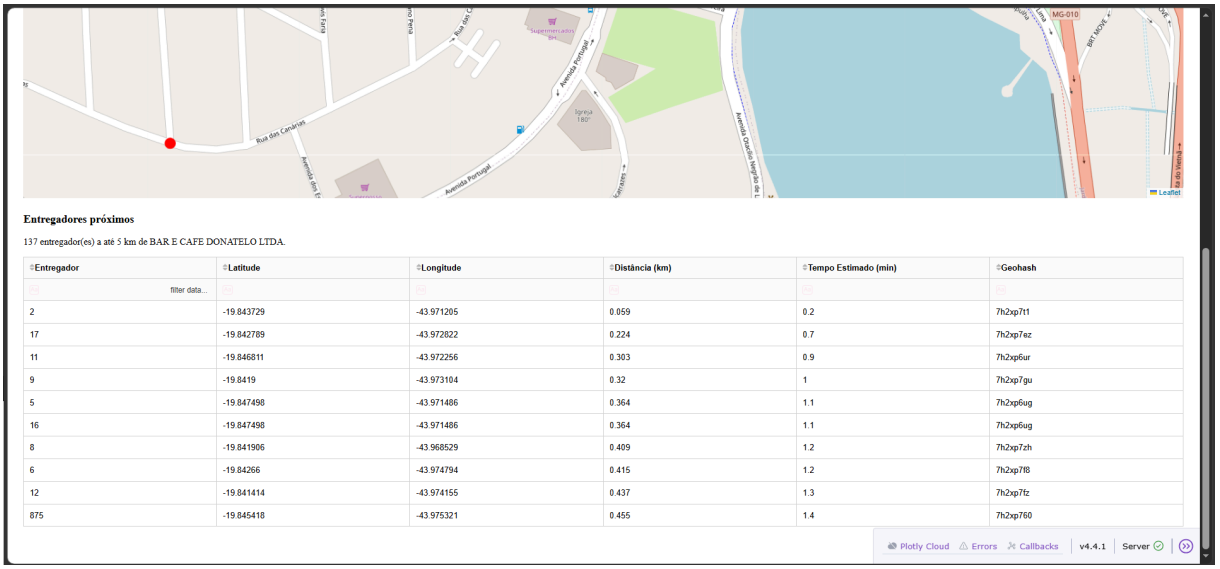
A operação mais básica que plataformas de entrega e transporte precisam realizar é descobrir quais entregadores estão mais próximos de um dado ponto de interesse. Como a resposta precisa ser imediata, a solução deve ser computacionalmente eficiente, o que exige métodos mais elaborados do que simplesmente calcular a distância até cada entregador da frota.

Este trabalho trata desse problema no contexto de uma frota de entregadores em operação na cidade de Belo Horizonte. O objetivo é implementar um motor de busca e monitoramento geoespacial híbrido, que combina uma estrutura de dados para strings com técnicas de geometria computacional. As posições dos entregadores são codificadas em Geohash e indexadas em uma Trie compacta, e os candidatos encontrados são filtrados pela distância até o restaurante.

Funcionamento do sistema e interface

O sistema é uma página web interativa feita com Dash e Dash Leaflet, bibliotecas que permitem montar mapas baseados no OpenStreetMap usando apenas Python. A página reúne em uma única tela os controles de busca, o mapa da cidade e uma tabela com os resultados.





Ao abrir a página, o sistema já gera uma frota inicial de entregadores. O usuário procura um restaurante pelo nome em um campo de busca, que filtra as opções conforme digita, e escolhe o raio de atendimento em um controle deslizante que vai de 1 a 20 km. Ao selecionar um restaurante, o mapa é ampliado e o sistema busca os entregadores que estão dentro do raio escolhido. Sempre que o usuário muda o restaurante ou o raio, a consulta é refeita e a página se atualiza automaticamente.

No mapa, cada restaurante aparece como um marcador azul e cada entregador como um círculo. Os entregadores dentro do raio ficam em vermelho e em destaque, e os de fora ficam em cinza, menores e mais transparentes. Ao passar o mouse sobre um pino, aparece o nome do restaurante ou, no caso de um entregador, seu identificador e seu Geohash.

Abaixo do mapa são mostradas informações da busca, como quantos entregadores foram encontrados dentro do range definido, distância, tempo estimado e Geohash.

A tabela já vem ordenada do mais próximo para o mais distante, e o usuário pode reordená-la ou filtrá-la por qualquer coluna.

A interface também permite gerar uma nova frota. O usuário informa a quantidade de entregadores e clica em “Gerar entregadores”. O sistema então descarta a frota anterior, esvazia a Trie e cria um novo conjunto do zero. Uma mensagem confirma quantos entregadores foram gerados.

A maior dificuldade aqui foi nos adaptarmos ao Dash, uma biblioteca que ainda não conhecíamos. A forma como os componentes se organizam e são atualizados pelos callbacks era diferente do que estávamos acostumados, e tivemos que relembrar o funcionamento de componentes HTML para fazer a interface funcionar como queríamos.

Arquitetura do sistema

Dividimos o sistema em poucos módulos, cada um com uma responsabilidade. O **main.py** monta a interface e contém os callbacks que respondem às ações do usuário. O **src/utils.py** reúne as funções ligadas ao Geohash, como a codificação das coordenadas e o cálculo das células vizinhas. A pasta **src/trie** contém nossa implementação da Trie compacta. O **src/osmn.py** cuida da geração dos entregadores e da correção das posições para as ruas.

Os dados de entrada são de dois tipos. Os restaurantes vêm de uma base de bares de Belo Horizonte, que tem apenas endereços. Por isso, convertemos esses endereços em coordenadas por geocodificação com o OSMnx e salvamos o resultado em um arquivo CSV. Os endereços que não puderam ser localizados foram descartados. O outro dado é a malha viária da cidade, que o OSMnx baixa do OpenStreetMap na primeira execução e guarda em um arquivo local para as execuções seguintes.

Quando a aplicação inicia, carregamos os restaurantes e geramos a frota. As posições sorteadas em torno de cada restaurante são levadas ao ponto mais próximo da malha viária. Em seguida, cada entregador tem sua posição codificada em um Geohash de 8 caracteres e é inserido na Trie. O botão de geração repete esse mesmo processo, esvaziando a Trie antes de inserir a nova frota.

Em uma consulta, o caminho é o inverso. A posição do restaurante escolhido é codificada em Geohash, e usamos os 5 primeiros caracteres para identificar a região onde ele está. Calculamos as oito células vizinhas dessa região e buscamos na Trie todos os entregadores das nove células. Esses candidatos passam então pelo cálculo de distância até o restaurante, e só os que estão dentro do raio seguem para a tabela e são destacados no mapa.

Implementação dos algoritmos e estruturas de dados

Codificação Geohash

Para gerar o Geohash, começamos com os intervalos completos de longitude, de -180 a 180, e de latitude, de -90 a 90. A cada passo, dividimos um dos intervalos ao meio, alternando entre longitude e latitude e começando pela longitude. Se a coordenada está na metade de cima, anotamos o bit 1 e ficamos com essa metade. Se está na metade de baixo, anotamos 0 e ficamos com a outra. A cada cinco bits, formamos um número de 0 a 31 e o convertemos para um caractere da tabela Base32.

Repetimos o processo até ter 8 caracteres, o limite pedido no enunciado, o que corresponde a uma célula de aproximadamente 38 por 19 metros. Cada entregador recebe o Geohash da sua posição, e é por essa string que ele é indexado na Trie.

Trie compacta

Cada nó da nossa Trie guarda três coisas. A primeira é um dicionário de filhos, em que a chave é o pedaço de string que leva até o filho. A segunda é uma marcação indicando se algum Geohash termina naquele nó. A terceira é uma lista com os entregadores daquele Geohash. Usamos uma lista porque vários entregadores podem ter o mesmo Geohash, o que é comum já que as posições são levadas aos mesmos pontos da malha viária.

Na inserção, comparamos o Geohash com as chaves dos filhos do nó atual. Se uma chave é inteiramente prefixo do Geohash, descemos para aquele filho e continuamos com o restante da string. Se a chave é igual ao que falta inserir, adicionamos o entregador à lista do nó. Se a chave e o Geohash têm só um pedaço em comum, quebramos a aresta em duas, criando um nó intermediário com a parte comum, que passa a ter como filhos o nó antigo e um novo nó para o entregador. Se nenhuma chave tem prefixo em comum, criamos um novo filho com todo o restante da string.

A busca por prefixo percorre a árvore da mesma forma, consumindo o prefixo aresta por aresta. Quando o prefixo acaba, seja em um nó ou no meio de uma aresta, recolhemos todos os entregadores da subárvore a partir dali. Encontrar esse ponto custa O(m), em que m é o tamanho do prefixo. Como em uma Trie compacta todo nó interno tem pelo menos dois filhos e toda folha guarda pelo menos um entregador, a subárvore tem menos nós do que o dobro de entregadores, e recolhê-los custa O(k). O total fica em O(m + k), como pede o enunciado.

Busca por prefixo e células adjacentes

Para buscar os entregadores de um restaurante, codificamos sua posição em Geohash e usamos os 5 primeiros caracteres como região. Em Belo Horizonte, uma célula desse tamanho tem cerca de 4,9 km de altura por 4,6 km de largura, o que corresponde aproximadamente à escala de um bairro.

Como um entregador muito próximo pode estar do outro lado da borda da célula, buscamos também as oito células vizinhas. Para encontrá-las, separamos os bits do Geohash de volta em dois números inteiros, um com os bits de longitude e outro com os de latitude, que indicam a posição da célula em uma grade. Somando ou subtraindo 1 desses números, chegamos às células ao redor, e intercalamos os bits novamente para obter seus Geohashes. Buscamos as nove células na Trie e reunimos os entregadores encontrados em um dicionário indexado pelo identificador, formando a lista de candidatos.

Esse cálculo e interação entre blocos vizinhos é consideravelmente abstrato, e entender como as células vizinhas se relacionam e trabalhar com operações bitwise para obter a melhor performance possível foi desafiador, porém genuinamente interessante.

Filtragem por distância e tempo de chegada

Para cada candidato, calculamos a distância até o restaurante. Usamos uma aproximação que trata a diferença de coordenadas como distância em um plano e converte graus para quilômetros multiplicando por 111. Os entregadores com distância menor ou igual ao raio são considerados dentro da área de atendimento, entram na tabela e aparecem em vermelho no mapa. Os demais aparecem em cinza. Como o raio vem do controle deslizante da interface, qualquer mudança nele refaz a filtragem.

Para estimar o tempo de chegada, supomos que o entregador se desloca em linha reta até o restaurante a uma velocidade média constante de 20 km/h, um valor razoável para motos e bicicletas no trânsito urbano. O tempo em minutos é a distância dividida pela velocidade, multiplicada por 60, e aparece em uma coluna da tabela. É uma estimativa simples que não considera o trajeto real pelas ruas nem o trânsito, complexidades do mundo real que não levamos em conta nesse trabalho para manter a objetividade no tema central.

Geração de dados sintéticos dos entregadores

Distribuição espacial baseada em demanda

A quantidade N de entregadores é informada pelo usuário na interface. Dividimos N igualmente entre os 58 restaurantes da base, descartando o resto da divisão, e por isso a frota gerada pode ter um pouco menos entregadores do que o pedido.

Em torno de cada restaurante, sorteamos as posições dos seus entregadores com uma distribuição normal centrada nas coordenadas dele, usando o NumPy. Sorteamos latitude e longitude de forma independente, com desvio padrão de 0,003 grau, o que equivale a pouco mais de 300 metros.

Com isso, a maior parte dos entregadores fica bem perto do restaurante, e a quantidade diminui aos poucos à medida que nos afastamos dele. Isso imita o comportamento real dos aplicativos, em que entregadores ociosos costumam esperar perto dos lugares com mais pedidos. Em regiões com vários restaurantes próximos, como o centro, as distribuições se sobrepõem e a densidade fica ainda maior.

Correção das posições para vias transitáveis

Como as posições são sorteadas matematicamente, muitas caem em lugares onde um entregador não poderia estar, como o meio de um quarteirão ou a Lagoa da Pampulha. Para corrigir isso, usamos o OSMnx para obter a malha viária de Belo Horizonte a partir do OpenStreetMap. A malha é um grafo em que os nós são pontos das ruas, como cruzamentos e curvas, e as arestas são os trechos entre eles. Como baixar o grafo leva tempo, salvamos ele em um arquivo na primeira execução e o carregamos desse arquivo nas seguintes.

Para cada posição sorteada, buscamos o nó do grafo mais próximo e substituímos a posição do entregador pelas coordenadas desse nó. Escolhemos usar os nós em vez do ponto mais próximo sobre a rua porque a busca é mais simples e o resultado já garante que todos os pinos fiquem sobre vias. A consequência é que vários entregadores podem acabar no mesmo cruzamento, e por isso a Trie guarda uma lista de entregadores em cada nó.

Essa parte não trouxe muita dificuldade. Na verdade, foi uma surpresa positiva ver como o OSMnx torna simples obter a malha de uma cidade inteira e encontrar o ponto de rua mais próximo de uma coordenada.

Regeneração dos dados

Quando o usuário informa uma nova quantidade e clica em “Gerar entregadores”, descartamos a frota anterior. A Trie é esvaziada trocando sua raiz por um nó novo, e então repetimos todo o processo de sorteio, correção e inserção. Por fim, os marcadores antigos são substituídos pelos novos no mapa e uma mensagem informa quantos entregadores foram gerados.

Experimentos

Executamos os experimentos em um computador com processador Intel Core i5-9400 de 2,90 GHz, 16 GB de memória RAM, sistema Windows 10 de 64 bits e Python 3.10.11. Os testes foram feitos pelo script experiments/experimentos.py, que gera a frota, monta a Trie e executa as consultas sem passar pela interface. Medimos o tempo com time.perf_counter, separando o tempo gasto na Trie do tempo total, que inclui a filtragem por distância. Cada consulta foi repetida 10 vezes e usamos a média. Para ter uma referência, medimos também uma busca linear, que calcula a distância do restaurante até todos os entregadores da frota. Fixamos a semente aleatória do NumPy para que a mesma frota fosse gerada a cada execução.

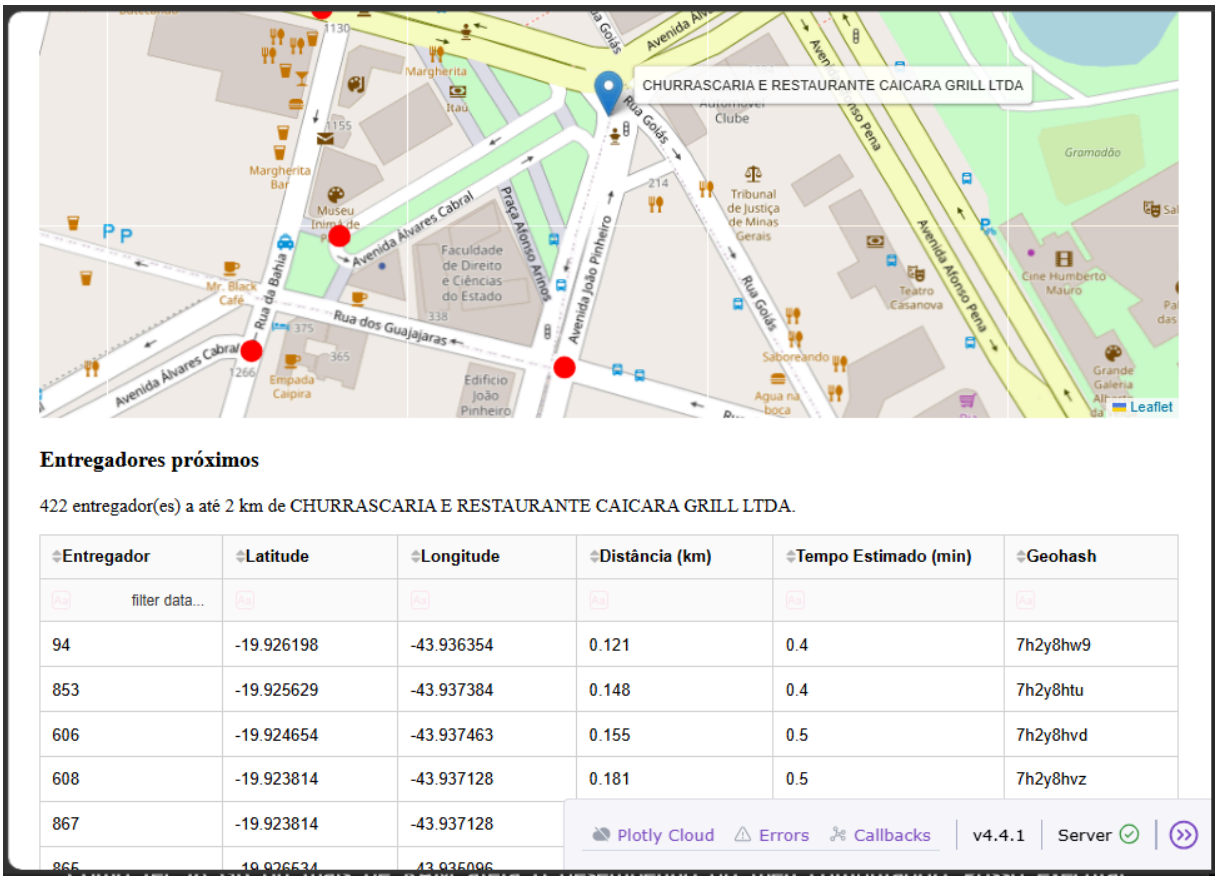
Escolhemos dois restaurantes com situações diferentes. A Churrascaria Caiçara Grill fica na região central, perto de muitos outros restaurantes e, portanto, de muitos entregadores. O Bar e Lanchonete Zum Zum fica mais afastado, com poucos restaurantes ao redor. Para cada um, variamos a frota entre 250, 1000 e 5000 entregadores e o raio entre 1, 2 e 4 km. Por causa da divisão inteira entre os 58 restaurantes, as frotas geradas tiveram 232, 986 e 4988 entregadores.

A tabela abaixo mostra os resultados das consultas. Os tempos são médias de 10 execuções, em milissegundos.

Restaurante	N	Raio (km)	Candidatos	No raio	Trie (ms)	Total (ms)	Linear (ms)
Caiçara Grill	232	1	199	36	0,253	0,376	0,153
Caiçara Grill	232	2	199	97	0,269	0,393	0,147
Caiçara Grill	232	4	199	158	0,245	0,370	0,140
Caiçara Grill	986	1	846	145	0,725	1,269	0,595
Caiçara Grill	986	2	846	427	0,697	1,230	0,600
Caiçara Grill	986	4	846	679	0,692	1,231	0,607
Caiçara Grill	4988	1	4278	706	2,450	5,093	2,926
Caiçara Grill	4988	2	4278	2143	2,374	5,011	2,978
Caiçara Grill	4988	4	4278	3405	2,700	5,620	3,617
Zum Zum	232	1	46	4	0,144	0,173	0,134

Zum Zum	232	2	46	4	0,145	0,177	0,134
Zum Zum	232	4	46	5	0,142	0,171	0,134
Zum Zum	986	1	205	17	0,263	0,390	0,579
Zum Zum	986	2	205	17	0,260	0,385	0,590
Zum Zum	986	4	205	17	0,259	0,384	0,588
Zum Zum	4988	1	1057	84	0,842	1,529	3,370
Zum Zum	4988	2	1057	86	0,785	1,459	3,199
Zum Zum	4988	4	1057	91	0,756	1,406	3,126

A figura abaixo mostra a consulta ao restaurante central com N = 1000 e raio de 2 km, com os entregadores em vermelho e cinza e a tabela de resultados.



Entregadores próximos

422 entregador(es) a até 2 km de CHURRASCARIA E RESTAURANTE CAICARA GRILL LTDA.

Entregador	Latitude	Longitude	Distância (km)	Tempo Estimado (min)	Geohash
filter data...					
94	-19.926198	-43.936354	0.121	0.4	7h2y8hw9
853	-19.925629	-43.937384	0.148	0.4	7h2y8htu
606	-19.924654	-43.937463	0.155	0.5	7h2y8hvd
608	-19.923814	-43.937128	0.181	0.5	7h2y8hvz
867	-19.923814	-43.937128			

Em todas as consultas, a busca pela Trie encontrou exatamente os mesmos entregadores que a busca linear. Isso confirma que, para raios de até 4 km, a busca nas células vizinhas não deixa ninguém de fora. Também vimos que o raio não muda

o número de candidatos, já que a região buscada é sempre a mesma, e por isso o tempo gasto na Trie praticamente não varia entre os raios.

O tempo da Trie acompanha o número de candidatos, e não o tamanho da frota. A consulta ao restaurante central com 986 entregadores e a consulta ao periférico com 4988 retornam quantidades parecidas de candidatos, 846 e 1057, e levam tempos parecidos na Trie, cerca de 0,7 e 0,8 ms, mesmo com frotas cinco vezes diferentes. Esse comportamento é coerente com o custo O(m + k) da busca por prefixo.

A tabela abaixo mostra o resumo por restaurante e tamanho de frota, com tempos médios dos três raios.

Restaurante	**N**	**Candidatos (% da frota)**	**Total Trie (ms)**	**Linear (ms)**	**Linear ÷ Trie**
Caiçara Grill	232	199 (86%)	0,380	0,147	0,39
Caiçara Grill	986	846 (86%)	1,244	0,601	0,48
Caiçara Grill	4988	4278 (86%)	5,241	3,174	0,61
Zum Zum	232	46 (20%)	0,174	0,134	0,77
Zum Zum	986	205 (21%)	0,386	0,586	1,52
Zum Zum	4988	1057 (21%)	1,465	3,232	2,21

Valores de “Linear ÷ Trie” acima de 1 indicam que a busca pela Trie foi mais rápida.

O ganho em relação à busca linear depende de quanto da frota a Trie consegue descartar. No restaurante periférico, as nove células buscadas contêm cerca de 20% da frota, e com 4988 entregadores a busca pela Trie fica cerca de 2,2 vezes mais rápida que a linear. No restaurante central, as nove células contêm cerca de 86% da frota, porque quase todos os restaurantes da base estão nessa região. Como quase nada é descartado, cada candidato passa duas vezes pelo trabalho, uma ao ser recolhido da Trie e outra na filtragem, enquanto a busca linear só calcula a distância uma vez. Nesse caso, a Trie acaba mais lenta. Com a frota de 232 entregadores, a busca linear é mais rápida nos dois restaurantes, porque o custo fixo de calcular os vizinhos e percorrer a árvore pesa mais quando há poucos entregadores.

Esses resultados mostram que um prefixo de 5 caracteres é grande demais para a região central da nossa base. Um prefixo maior descartaria mais entregadores, mas cobriria um raio menor, o que reforça a ideia de escolher o tamanho do prefixo de acordo com o raio pedido.

Conclusão

Os experimentos mostraram que a busca encontra exatamente os mesmos entregadores que uma busca linear e que seu tempo depende do número de candidatos, e não do tamanho da frota. Em regiões com menos entregadores, a Trie descarta cerca de 80% da frota e a consulta chega a ser duas vezes mais rápida que a busca linear. Na região central, porém, quase toda a frota cai nas nove células buscadas, e a Trie perde para a busca linear.

O sistema tem algumas limitações. Como usamos sempre um prefixo de 5 caracteres, a região buscada cobre com segurança apenas raios de até cerca de 4,6 km, e raios maiores podem deixar entregadores de fora. Na região central, esse mesmo prefixo é grande demais e descarta pouco da frota. A distância é calculada por uma aproximação plana, que superestima um pouco as distâncias no sentido leste e oeste. O tempo de chegada considera um trajeto em linha reta a velocidade constante, sem levar em conta as ruas e o trânsito. Na simulação, o resto da divisão de N entre os restaurantes é descartado, e os entregadores são levados ao nó mais próximo da malha viária, o que faz vários deles ocuparem o mesmo ponto. Essas questões poderiam ser tratadas em versões futuras, por exemplo ajustando o tamanho do prefixo ao raio e calculando o tempo pela rota real no grafo de ruas.

Apesar disso, o sistema atende aos objetivos do trabalho e mostra, na prática, como uma estrutura de dados para strings pode tornar uma busca geográfica mais eficiente, desde que a região buscada descarte boa parte da frota.

Referências

ANWARVIC. Radix Trie. In: extra-collections 1.0.0 documentation. [S. l.], 2020. Disponível em: https://extra-collections.readthedocs.io/en/latest/rst/trees/radix_trie.html. Acesso em: 20 out. 2026.

BOEING, G. OSMnx: new methods for acquiring, constructing, analyzing, and visualizing complex street networks. Computers, Environment and Urban Systems, v. 65, p. 126-139, 2017. DOI: 10.1016/j.compenvurbsys.2017.05.004.

DASH-LEAFLET. In: PyPI: the Python Package Index. [S. l.]: Python Software Foundation, [20--]. Disponível em: https://pypi.org/project/dash-leaflet/. Acesso em: 20 out. 2026.

GEOHASH. In: Wikipedia: the free encyclopedia. [San Francisco, CA]: Wikimedia Foundation, 2026. Disponível em: https://en.wikipedia.org/wiki/Geohash. Acesso em: 20 out. 2026.

HARRIS, C. R. et al. Array programming with NumPy. Nature, v. 585, n. 7825, p. 357-362, 2020. DOI: 10.1038/s41586-020-2649-2.

OPENSTREETMAP CONTRIBUTORS. OpenStreetMap. Disponível em: https://www.openstreetmap.org. Acesso em: 20 out. 2026.

PLOTLY. Dash documentation. Disponível em: https://dash.plotly.com. Acesso em: 20 out. 2026.

PLOTLY. Determining which callback input changed. In: Plotly Dash documentation. [S. l.], [20--]. Disponível em: https://dash.plotly.com/determining-which-callback-input-changed. Acesso em: 20 out. 2026.

PREFEITURA DE BELO HORIZONTE. Atividade econômica. Portal de Dados Abertos da PBH. Disponível em: https://dados.pbh.gov.br/dataset/atividade-economica. Acesso em: 16 out. 2026.

RADIX TREE. In: Wikipedia: the free encyclopedia. [San Francisco, CA]: Wikimedia Foundation, 2026. Disponível em: https://en.wikipedia.org/wiki/Radix_tree. Acesso em: 20 out. 2026.

THE PANDAS DEVELOPMENT TEAM. pandas. Disponível em: https://pandas.pydata.org. Acesso em: 20 out. 2026.

VIMIEIRO, R. Trabalho Prático 1: indexação geoespacial e geometria computacional híbrida. Belo Horizonte: Departamento de Ciência da Computação, UFMG, 2026. Enunciado da disciplina DCC207, Algoritmos 2.